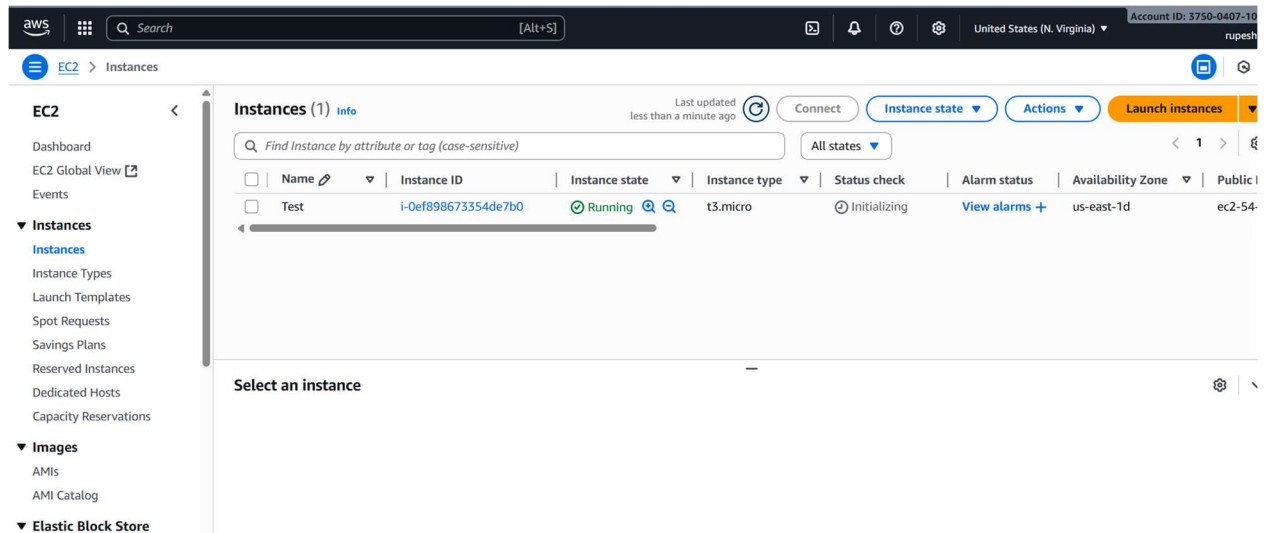


Backend Flask + AWS RDS Integration

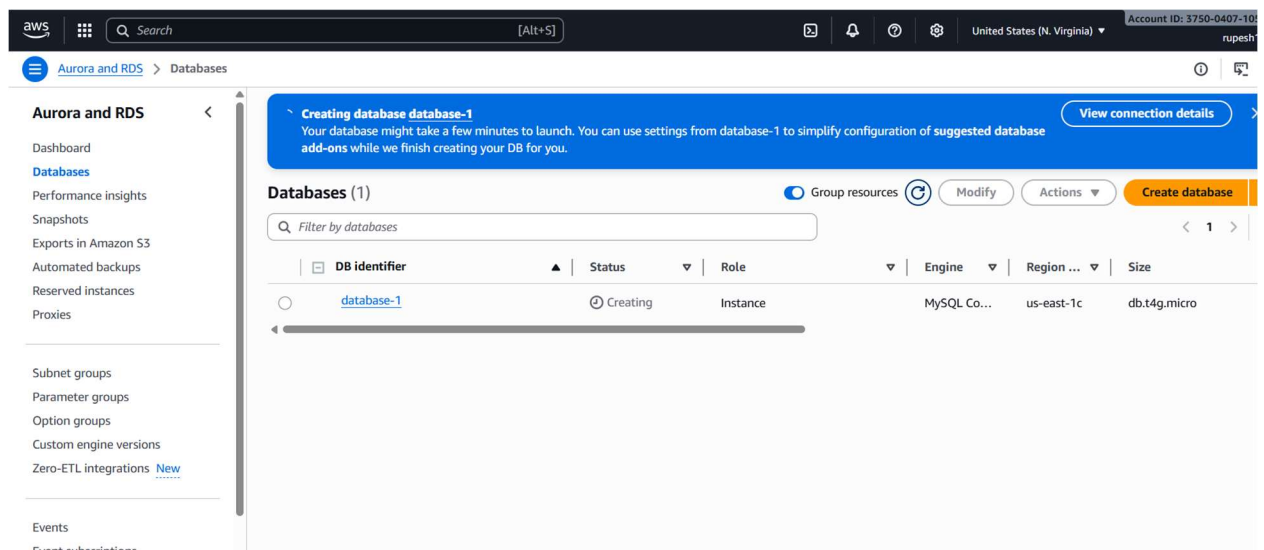
Step 1: Launch EC2 Instance

- Launch EC2 with a Security Group → **allow all traffic (for testing only)**.



Step 2: Create RDS

- Open AWS RDS and create a DB instance.
- Configure **Security Group**



- Connect via **Ec2 instance**
- Create database & table:

```
CREATE DATABASE dev;
USE dev;
```

```
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE
);
```

```
[root@ip-172-31-24-195 ~]# mysql -h database-1.cqdi4o0cc9ww.us-east-1.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 29
Server version: 8.0.42 Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]> create database dev;
Query OK, 1 row affected (0.015 sec)

MySQL [(none)]> use dev;
Database changed
MySQL [dev]> CREATE TABLE users (
->     id INT AUTO_INCREMENT PRIMARY KEY,
->     name VARCHAR(100) NOT NULL,
->     email VARCHAR(100) NOT NULL UNIQUE
-> );
Query OK, 0 rows affected (0.036 sec)

MySQL [dev]> █
```

Step 3: Install Dependencies

```
sudo yum install -y python3-pip
pip3 install flask mysql-connector-python
pip3 install flask_cors
```

```
Collecting Jinja2>=3.1.2
  Downloading Jinja2-3.1.6-py3-none-any.whl (134 kB)
    |#####| 134 kB 28.9 MB/s
Collecting blinker>=1.9.0
  Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Collecting importlib-metadata>=3.6.0
  Downloading importlib_metadata-8.7.0-py3-none-any.whl (27 kB)
Collecting click>=8.1.3
  Downloading click-8.1.8-py3-none-any.whl (98 kB)
    |#####| 98 kB 17.1 MB/s
Collecting MarkupSafe>=2.1.1
  Downloading MarkupSafe-3.0.2-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (20 kB)
Collecting zipp>=3.20
  Downloading zipp-3.23.0-py3-none-any.whl (10 kB)
Installing collected packages: zipp, MarkupSafe, werkzeug, Jinja2, itsdangerous, importlib-metadata, click, blinker, mysql-connector-python, flask
Attempting uninstall: MarkupSafe
  Found existing installation: MarkupSafe 1.1.1
  Uninstalling MarkupSafe-1.1.1:
    Successfully uninstalled MarkupSafe-1.1.1
Attempting uninstall: Jinja2
  Found existing installation: Jinja2 2.11.3
  Uninstalling Jinja2-2.11.3:
    Successfully uninstalled Jinja2-2.11.3
Successfully installed blinker-1.9.0 click-8.1.8 flask-3.1.2 importlib-metadata-8.7.0 itsdangerous-2.2.0 Jinja2-3.1.6 MarkupSafe-3.0.2 mysql-connector-python-9.4.0 werkzeug-3.1.3 zipp-3.23.0
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv
[root@ip-172-31-24-195 ~]# █
```

Step 4: Install Git & Clone Project

```
sudo yum install -y git
git clone https://github.com/CloudTechDevOps/python-backend-testing.git
cd python-backend-testing/
ls    # should see app.py
```

```
Attempting uninstall: Jinja2
Found existing installation: Jinja2 2.11.3
Uninstalling Jinja2-2.11.3:
  Successfully uninstalled Jinja2-2.11.3
Successfully installed blinker-1.9.0 click-8.1.8 flask-3.1.2 importlib-metadata-8.7.0 itsdangerous-2.2.0 Jinja2-3.1.6 markupsafe-3.0.2 mysql-connector-python-9.4.0 Werkzeug-3.1.3 zipp-3.23.0
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv
[root@ip-172-31-24-195 ~]# yum install git
Last metadata expiration check: 0:13:54 ago on Mon Sep 22 11:40:53 2025.
Dependencies resolved.
=====
Package                        Architecture      Version           Repository        Size
-----
Installing:
git                            x86_64            2.50.1-1.amzn2023.0.1  amazonlinux      53 k
Installing dependencies:
git-core                      x86_64            2.50.1-1.amzn2023.0.1  amazonlinux      4.9 M
git-core-doc                  noarch            2.50.1-1.amzn2023.0.1  amazonlinux      2.8 M
perl-Error                    noarch            1:0.17029-5.amzn2023.0.2  amazonlinux      41 k
perl-File-Find                noarch            1.37-477.amzn2023.0.7    amazonlinux      25 k
perl-Git                      noarch            2.50.1-1.amzn2023.0.1    amazonlinux      41 k
perl-TermReadKey              x86_64            2.38-9.amzn2023.0.2     amazonlinux      36 k
perl-lib                      x86_64            0.65-477.amzn2023.0.7    amazonlinux      15 k
Transaction Summary
-----
Install 8 Packages
```

Step 5: Configure Database in app.py

Edit the file:

```
# Database Configuration
db_config = {
    'host': 'database-1.cqdi4o0cc9ww.us-east-1.rds.amazonaws.com',
    'user': 'admin',
    'password': 'SkyOps1234',
    'database': 'dev'
}
```

```
from flask import Flask, request, jsonify
from flask_cors import CORS
import mysql.connector

app = Flask(__name__)
CORS(app)  # Enables frontend-to-backend access

# Database Configuration
db_config = {
    'host': 'database-1.cqdi4o0cc9ww.us-east-1.rds.amazonaws.com',
    'user': 'admin',
    'password': 'SkyOps1234',
    'database': 'dev'
}

def get_db_connection():
    return mysql.connector.connect(**db_config)

# Get all users
@app.route('/users', methods=['GET'])
def get_users():
    conn = get_db_connection()
    cursor = conn.cursor(dictionary=True)
    cursor.execute("SELECT * FROM users")
    users = cursor.fetchall()
    cursor.close()
    conn.close()
```

Step 6: Run app.py

- Run in foreground:

```
python3 app.py
```

Output:

```
* Running on http://0.0.0.0:5000
```

Stop with CTRL+C.

```
[root@ip-172-31-24-195 python-backend-testing]# python3 app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.31.24.195:5000
Press CTRL+C to quit
```

- Run in background:

```
nohup python3 app.py > flask.log 2>&1 &
```

nohup → keeps running after logout

& → background execution

flask.log → logs output

```
^C[root@ip-172-31-24-195 python-backend-testing]# nohup python3 app.py > flask.log 2>&1 &
[1] 28204
[root@ip-172-31-24-195 python-backend-testing]#
```

Step 7: Test APIs (using curl)

Run in another terminal:

- POST → Add user

```
curl -X POST http://localhost:5000/users/add \
-H "Content-Type: application/json" \
-d '{"name": "John Doe", "email": "john@example.com"}'
```

```
[ec2-user@ip-172-31-24-195 ~]$ curl -X POST http://localhost:5000/users/add \
-H "Content-Type: application/json" \
-d '{"name": "John Doe", "email": "john@example.com"}'
{"message": "User added successfully"}
[ec2-user@ip-172-31-24-195 ~]$
```

- **GET → Fetch all users**

```
curl -X GET http://localhost:5000/users
```

```
[ec2-user@ip-172-31-24-195 ~]$ curl -X GET http://localhost:5000/users
[{"email":"john@example.com","id":1,"name":"John Doe"}]
[ec2-user@ip-172-31-24-195 ~]$
```

- **GET → Fetch single user**

```
curl -X GET http://localhost:5000/users/1
```

```
[ec2-user@ip-172-31-24-195 ~]$ curl -X GET http://localhost:5000/users/1
{"email":"john@example.com","id":1,"name":"John Doe"}
[ec2-user@ip-172-31-24-195 ~]$
```

- **PUT → Update user**

```
curl -X PUT http://localhost:5000/users/update/1 \
-H "Content-Type: application/json" \
-d '{"name": "John Updated", "email": "john.updated@example.com"}'
```

```
[ec2-user@ip-172-31-24-195 ~]$ curl -X PUT http://localhost:5000/users/update/1 \
-H "Content-Type: application/json" \
-d '{"name": "John Updated", "email": "john.updated@example.com"}'
{"message":"User updated successfully"}
[ec2-user@ip-172-31-24-195 ~]$ curl -X GET http://localhost:5000/users/1
{"email":"john.updated@example.com","id":1,"name":"John Updated"}
[ec2-user@ip-172-31-24-195 ~]$
```

- **DELETE → Remove user**

```
curl -X DELETE http://localhost:5000/users/delete/1
```

```
[ec2-user@ip-172-31-24-195 ~]$ curl -X DELETE http://localhost:5000/users/delete/1
{"message":"User deleted successfully"}
[ec2-user@ip-172-31-24-195 ~]$ curl -X GET http://localhost:5000/users
[]
[ec2-user@ip-172-31-24-195 ~]$
```